

PyQuiz

Dokumentacja Projektu

Personalna aplikacja do nauki Python + Django
z algorytmem Spaced Repetition (SM-2)

Python

Django

Next.js

PostgreSQL

Docker

Wersja 1.0 · Kwiecień 2026 · Krystian

Spis treści

1	Opis projektu	str. 3
2	Cele i założenia	str. 3
3	Architektura systemu	str. 4
4	Schemat bazy danych	str. 5
5	Backend — Django REST API	str. 9
6	Frontend — Next.js	str. 11
7	Algorytm SRS (SM-2)	str. 13
8	System importu pytań	str. 15
9	Typy pytań i quizów	str. 16
10	Dashboard i analityka	str. 18
11	Mapa ekranów	str. 20
12	Docker i środowisko lokalne	str. 21
13	Struktura projektu	str. 22
14	Plan implementacji	str. 23
15	Format JSON pytań	str. 25
16	API Endpoints	str. 27
17	Słownik pojęć	str. 29

1. Opis projektu

PyQuiz to zaawansowana, personalna aplikacja do nauki i przygotowania do egzaminu końcowego bootcampu Python + Django. Aplikacja umożliwia systematyczne przyswajanie wiedzy poprzez quizy z materiałów kursowych dostarczanych w formacie PDF, z wykorzystaniem algorytmu powtórek rozłożonych w czasie (Spaced Repetition System — SRS).

Pytania do każdej lekcji są generowane przez Claude AI na podstawie PDF-ów z wykładami i importowane do bazy danych aplikacji. Użytkownik ma pełną kontrolę nad trybem nauki, trudnością pytań, zakresem materiału i tempem nauki.

Kluczowa zasada: Pytania nie są generowane automatycznie przez zintegrowane API — Claude AI generuje je zewnątrz w formacie JSON, który następnie importujesz do aplikacji przez skrypt CLI lub panel w interfejsie. To daje pełną kontrolę nad jakością pytań.

2. Cele i założenia

Główne cele

- Skuteczne przygotowanie do egzaminu końcowego bootcampu
- Systematyczna nauka z wykorzystaniem algorytmu SM-2
- Identyfikacja słabych obszarów i ukierunkowana powtórka
- Pełna przejrzystość postępów i analityka nauki
- Profesjonalne, przyjemne w użyciu narzędzie do codziennej nauki

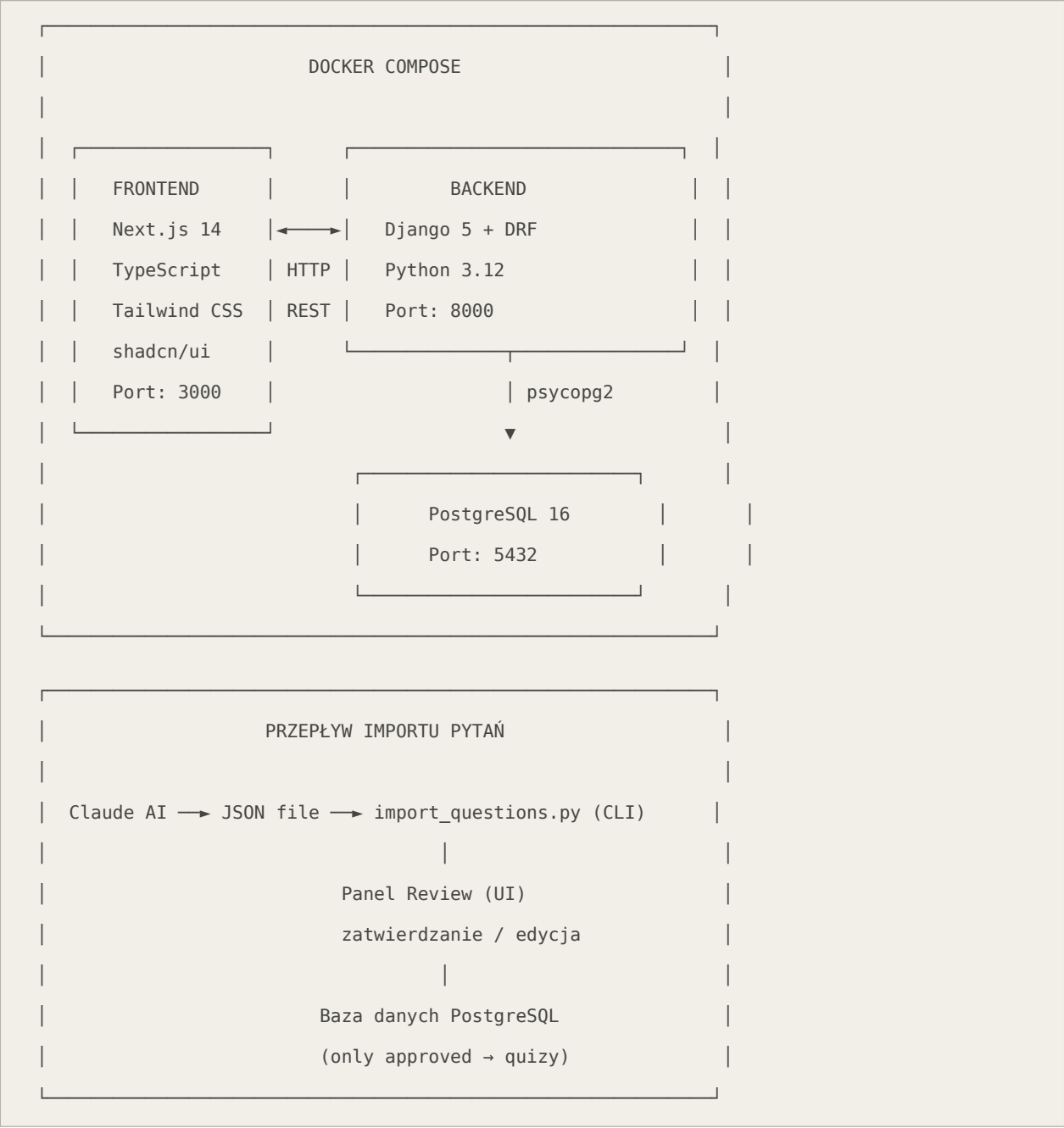
Założenia projektowe

Parametr	Wartość
Liczba lekcji	30-80
Język aplikacji	Polski
Środowisko uruchomieniowe	Lokalnie (Docker)
Liczba użytkowników	1 (architektura multi-user od razu)
Źródło pytań	Claude AI → JSON → import
Typy pytań	MCQ (jedno/wielokrotny wybór) + Otwarte + Kod
Trudności pytań	easy / medium / hard / expert

3. Architektura systemu

Aplikacja zbudowana jest jako klasyczna architektura klient-serwer z wyraźnym podziałem na frontend (Next.js), backend (Django REST API) i bazę danych (PostgreSQL), wszystko opakowane w kontenery Docker.

Diagram architektury



Stos technologiczny

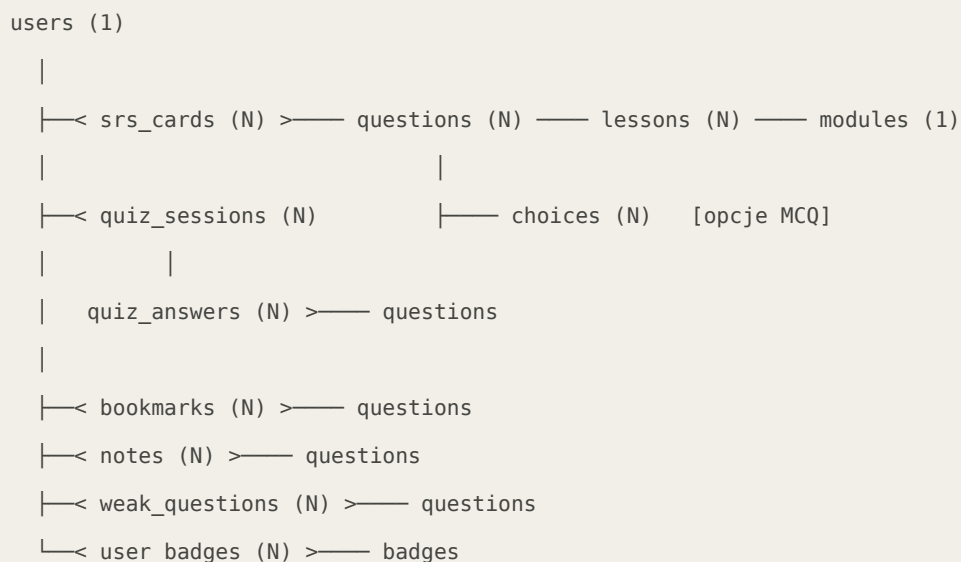
Warstwa	Technologia	Wersja	Zastosowanie
Backend	Django	5.x	Framework webowy
Backend	Django REST Framework	3.15	API REST
Backend	psycopg2	2.9	Sterownik PostgreSQL

Warstwa	Technologia	Wersja	Zastosowanie
Backend	pdfplumber	0.11	Ekstrakcja tekstu z PDF
Baza danych	PostgreSQL	16	Relacyjna baza danych
Frontend	Next.js	14 (App Router)	Framework React
Frontend	TypeScript	5.x	Typowanie statyczne
Frontend	Tailwind CSS	3.x	Stylowanie
Frontend	shadcn/ui	latest	Komponenty UI
Frontend	Recharts	2.x	Wykresy i wizualizacje
Frontend	react-pdf	latest	Podgląd plików PDF
DevOps	Docker + Compose	latest	Konteneryzacja

4. Schemat bazy danych

Baza danych składa się z 13 tabel podzielonych na cztery obszary funkcjonalne: materiał kursowy, bank pytań, sesje quizów i SRS, oraz funkcje pomocnicze (zakładki, notatki, odznaki, streak).

Diagram relacji (ERD)



Tabele i kolumny

modules — Moduły kursu

Kolumna	Typ	Opis
id	SERIAL PK	Klucz główny
number	INTEGER UNIQUE	Numer kolejny modułu
title	VARCHAR(200)	Tytuł modułu
description	TEXT	Opis modułu
created_at	TIMESTAMP	Data utworzenia

lessons — Lekcje

Kolumna	Typ	Opis
id	SERIAL PK	Klucz główny
module_id	FK → modules	Przynależność do modułu
number	INTEGER UNIQUE	Numer globalny lekcji
title	VARCHAR(200)	Tytuł lekcji

Kolumna	Typ	Opis
pdf_path	VARCHAR(500)	Ścieżka do pliku PDF
summary	TEXT	Streszczenie lekcji (z Claude)
created_at	TIMESTAMP	Data utworzenia

questions — Bank pytań

Kolumna	Typ	Opis
id	SERIAL PK	Klucz główny
lesson_id	FK → lessons	Przynależność do lekcji
type	VARCHAR(10)	MCQ / OPEN / CODE
difficulty	VARCHAR(10)	easy / medium / hard / expert
question_text	TEXT	Treść pytania
correct_answer	TEXT	Wzorcowa odpowiedź
explanation	TEXT	Wyjaśnienie po odpowiedzi
topic_tags	TEXT[]	Tagi tematyczne (tablica)
is_approved	BOOLEAN	Czy zatwierdzone przez review
multi_correct	BOOLEAN	Czy MCQ wielokrotny wybór
source_context	TEXT	Fragment PDF źródłowy
created_at	TIMESTAMP	Data utworzenia

choices — Opcje MCQ

Kolumna	Typ	Opis
id	SERIAL PK	Klucz główny
question_id	FK → questions	Pytanie nadrzędne (CASCADE DELETE)
label	CHAR(1)	Etykieta: a, b, c, d
text	TEXT	Treść opcji
is_correct	BOOLEAN	Czy poprawna odpowiedź

srs_cards — Stan algorytmu SM-2

Kolumna	Typ	Opis
id	SERIAL PK	Klucz główny
user_id	FK → users	Użytkownik

Kolumna	Typ	Opis
question_id	FK → questions	Pytanie
interval	INTEGER	Dni do następnej powtórki (domyślnie 1)
repetitions	INTEGER	Liczba poprawnych powtórzeń z rzędu
ease_factor	FLOAT	Współczynnik łatwości EF (domyślnie 2.5)
next_review_date	DATE	Data następnej zaplanowanej powtórki
last_reviewed_at	TIMESTAMP	Ostatnia powtórka

quiz_sessions — Sesje quizów

Kolumna	Typ	Opis
id	SERIAL PK	Klucz główny
user_id	FK → users	Użytkownik
mode	VARCHAR(20)	manual / srs / exam / flashcard / mistakes
filters	JSONB	Konfiguracja quizu (lekcje, trudność, itp.)
timer_seconds	INTEGER	Limit czasu w sekundach (nullable)
started_at	TIMESTAMP	Czas rozpoczęcia
finished_at	TIMESTAMP	Czas zakończenia (nullable)
score	INTEGER	Liczba poprawnych odpowiedzi
total	INTEGER	Łączna liczba pytań
percentage	FLOAT	Wynik procentowy

quiz_answers — Odpowiedzi w sesji

Kolumna	Typ	Opis
id	SERIAL PK	Klucz główny
session_id	FK → quiz_sessions	Sesja quizu
question_id	FK → questions	Pytanie
given_answers	JSONB	Udzielone odpowiedzi (np. ["a","c"])
is_correct	BOOLEAN	Czy odpowiedź poprawna
is_partial	BOOLEAN	Czy częściowo poprawna (multi-correct MCQ)
self_rating	VARCHAR(10)	Samoocena: good / partial / bad
time_spent_sec	INTEGER	Czas odpowiedzi w sekundach
skipped	BOOLEAN	Czy pytanie pominięte

weak_questions — Do poprawy

Kolumna	Typ	Opis
id	SERIAL PK	Klucz główny
user_id	FK → users	Użytkownik
question_id	FK → questions	Pytanie
fail_count	INTEGER	Liczba błędnych odpowiedzi
last_failed_at	TIMESTAMP	Ostatni błąd
is_resolved	BOOLEAN	Czy opanowane

5. Backend — Django REST API

Struktura aplikacji Django

```
backend/
├── config/                # Konfiguracja projektu
│   ├── settings/
│   │   ├── base.py       # Wspólne ustawienia
│   │   └── local.py      # Ustawienia lokalne (DEBUG=True)
│   ├── urls.py           # Główny routing URL
│   └── wsgi.py
├── apps/
│   ├── users/            # Użytkownicy i autoryzacja
│   ├── lessons/         # Moduły, lekcje, obsługa PDF
│   ├── questions/       # Bank pytań, import, review
│   ├── quiz/            # Sesje quizów, odpowiedzi
│   ├── srs/             # Algorytm SM-2, karty SRS
│   └── analytics/       # Dashboard, statystyki, raporty
├── scripts/
│   └── import_questions.py # Skrypt CLI do importu JSON
└── manage.py
```

Przykładowy model — Question

```
# apps/questions/models.py
from django.db import models
from django.contrib.postgres.fields import ArrayField

class Question(models.Model):
    TYPE_CHOICES = [
        ('MCQ', 'Multiple Choice'),
        ('OPEN', 'Open'),
        ('CODE', 'Code')
    ]
    DIFFICULTY_CHOICES = [
        ('easy', 'Łatwe'), ('medium', 'Średnie'),
        ('hard', 'Trudne'), ('expert', 'Expert')
    ]
```

```
lesson      = models.ForeignKey('lessons.Lesson',
                                on_delete=models.CASCADE, related_name='questions')

type        = models.CharField(max_length=10, choices=TYPE_CHOICES)
difficulty  = models.CharField(max_length=10, choices=DIFFICULTY_CHOICES)
question_text= models.TextField()
correct_answer= models.TextField(blank=True)
explanation   = models.TextField(blank=True)
topic_tags   = ArrayField(models.CharField(max_length=100), default=list)
is_approved  = models.BooleanField(default=False)
multi_correct= models.BooleanField(default=False)
source_context= models.TextField(blank=True)
created_at   = models.DateTimeField(auto_now_add=True)

class Meta:
    ordering = ['lesson', 'difficulty', 'id']
```

Przykładowy widok — Quiz Session

```
# apps/quiz/views.py

from rest_framework import viewsets, status
from rest_framework.decorators import action
from rest_framework.response import Response

class QuizSessionViewSet(viewsets.ModelViewSet):
    queryset = QuizSession.objects.all()
    serializer_class = QuizSessionSerializer

    def create(self, request):
        """Tworzenie nowej sesji quizu z filtrowaniem pytań."""
        filters = request.data
        questions = self._build_question_set(filters)
        session = QuizSession.objects.create(
            user=request.user,
            mode=filters.get('mode', 'manual'),
            filters=filters,
            timer_seconds=filters.get('timer_seconds'),
            total=len(questions)
        )
        session.questions.set(questions)
        return Response(QuizSessionSerializer(session).data,
                        status=status.HTTP_201_CREATED)

    @action(detail=True, methods=['post'])
    def answer(self, request, pk=None):
```

```
"""Zapisanie odpowiedzi i aktualizacja SRS."""  
session = self.get_object()  
answer = self._process_answer(session, request.data)  
self._update_srs(request.user, answer)  
return Response(QuizAnswerSerializer(answer).data)
```

6. Frontend — Next.js

Struktura katalogów Next.js 14 (App Router)

```
frontend/  
├─ app/  
│   ├─ layout.tsx          # Root layout (sidebar + topbar)  
│   ├─ dashboard/page.tsx  # Dashboard główny  
│   ├─ quiz/  
│   │   ├─ configure/page.tsx # Konfiguracja quizu  
│   │   └─ [sessionId]/  
│   │       ├─ page.tsx      # Aktywny quiz  
│   │       └─ summary/page.tsx  
│   ├─ lessons/  
│   │   ├─ page.tsx          # Lista modułów i lekcji  
│   │   └─ [id]/page.tsx     # Szczegóły lekcji + PDF viewer  
│   ├─ review/page.tsx      # Panel review pytań  
│   ├─ mistakes/page.tsx    # Do poprawy  
│   ├─ bookmarks/page.tsx  
│   ├─ notes/page.tsx  
│   ├─ progress/page.tsx    # Analityka  
│   ├─ exam/page.tsx        # Tryb egzaminacyjny  
│   └─ settings/page.tsx  
│  
├─ components/  
│   ├─ layout/Sidebar.tsx  
│   ├─ quiz/  
│   │   ├─ MCQQuestion.tsx  
│   │   ├─ OpenQuestion.tsx  
│   │   ├─ CodeQuestion.tsx  
│   │   └─ QuizProgress.tsx  
│   └─ dashboard/  
│       ├─ KnowledgeMap.tsx  
│       └─ WeeklyChart.tsx  
│  
└─ lib/  
    ├─ api.ts               # Axios klient  
    └─ types.ts             # Typy TypeScript
```

Kluczowe typy TypeScript

```
// lib/types.ts

export type Difficulty = 'easy' | 'medium' | 'hard' | 'expert';
export type QuestionType = 'MCQ' | 'OPEN' | 'CODE';
export type QuizMode = 'manual' | 'srs' | 'exam' | 'flashcard' | 'mistakes';
export type SelfRating = 'good' | 'partial' | 'bad';

export interface Question {
  id: number;
  lessonId: number;
  lessonTitle: string;
  type: QuestionType;
  difficulty: Difficulty;
  questionText: string;
  choices?: Choice[];
  correctAnswer: string;
  explanation: string;
  topicTags: string[];
  multiCorrect: boolean;
  sourceContext?: string;
}

export interface QuizSession {
  id: number;
  mode: QuizMode;
  questions: Question[];
  currentIndex: number;
  answers: QuizAnswer[];
  startedAt: string;
  timerSeconds?: number;
}
```

7. Algorytm SRS (SM-2)

PyQuiz używa algorytmu SuperMemo 2 (SM-2) do planowania powtórek. Algorytm dostosowuje interwały między powtórkami na podstawie oceny użytkownika po każdej odpowiedzi — pytania dobrze opanowane są pokazywane rzadziej, a trudne częściej.

Parametry algorytmu

Parametr	Znaczenie	Wartość domyślna
interval	Liczba dni do następnej powtórki	1
repetitions	Liczba poprawnych powtórzeń z rzędu	0
ease_factor (EF)	Współczynnik łatwości — im wyższy, tym rzadziej pokazywane	2.5

Skala ocen

Wynik	Znaczenie	Wartość q
☐ Nie wiedziałem (bad)	Zupełnie błędna odpowiedź	0
⚠ Częściowo (partial)	Częściowo poprawna odpowiedź	2
☑ Wiedziałem (good)	Poprawna odpowiedź	5
MCQ — poprawna	Automatyczna ocena	5
MCQ — błędna	Automatyczna ocena	0

Implementacja Python (SM-2)

```
# srs/algorithm.py

from datetime import date, timedelta
from dataclasses import dataclass

@dataclass
class SM2Result:
    interval: int
    repetitions: int
    ease_factor: float
    next_review_date: date

def calculate_sm2(quality: int, repetitions: int,
                  ease_factor: float, interval: int) -> SM2Result:
    if quality >= 3:
        if repetitions == 0:
            new_interval = 1
```



```

        elif repetitions == 1: new_interval = 6
        else:
            new_interval = round(interval * ease_factor)
        new_repetitions = repetitions + 1
        new_ef = ease_factor + (0.1 - (5-quality)*(0.08 + (5-quality)*0.02))
        new_ef = max(1.3, new_ef)
    else:
        new_interval = 1
        new_repetitions = 0
        new_ef = ease_factor # EF nie zmienia się przy błędzie

    return SM2Result(
        interval = new_interval,
        repetitions = new_repetitions,
        ease_factor = round(new_ef, 2),
        next_review_date = date.today() + timedelta(days=new_interval)
    )

```

Przepływ SRS — diagram

Użytkownik odpowiada na pytanie

|

▼

Typ pytania MCQ?

├ Tak → Poprawna? → q=5 Błędna? → q=0

└ Nie (OPEN/CODE) → Samoocena użytkownika:

 good → q=5

 partial → q=2

 bad → q=0

|

▼

Uruchom calculate_sm2(q, repetitions, ease_factor, interval)

|

├ Zapisz zaktualizowany SRSCard do bazy

|

└ q < 3? → Dodaj/aktualizuj rekord w weak_questions

8. System importu pytań

Workflow importu krok po kroku

- 1 Kończysz lekcję na bootcampie i pobierasz PDF z materiałami
- 2 Wysyłasz PDF do Claude z kontekstem: "Lekcja [N] — [Tytuł]. Moduł: [Tytuł modułu]"
- 3 Claude analizuje materiał i generuje plik JSON z pytaniami, opcjami, wyjaśnieniami i streszczeniem lekcji
- 4 Zapisujesz wygenerowany JSON do katalogu /imports/ w projekcie
- 5 Import przez CLI: `python import_questions.py --file lekcja_12.json`
- 6 LUB import przez panel w aplikacji: przejdź do /review, kliknij "Importuj JSON", wklej lub prześlij plik
- 7 Pytania trafiają do kolejki REVIEW (`is_approved=False`)
- 8 W panelu review przeglądasz pytania lekcji, zatwierdzasz całość lub edytujesz/odrzucaś pojedyncze
- 9 Zatwierdzone pytania są dostępne we wszystkich trybach quizów i SRS

Skrypt CLI — użycie

```
# Podstawowy import (pytania trafiają do kolejki review)
python import_questions.py --file lekcja_12.json

# Import z automatycznym zatwierdzeniem wszystkich pytań
python import_questions.py --file lekcja_12.json --approve-all

# Import całego katalogu naraz
python import_questions.py --dir ./imports/

# Przykładowy output:
# □ Lekcja 12: Dekoratory w Pythonie
#   Zaimportowano: 22 pytania
#   Pominęto (duplikaty): 0 pytań
#   Status: OCZEKUJE NA REVIEW w panelu aplikacji
```

9. Typy pytań i quizów

Typy pytań

Typ	Opis	Ocenianie	SRS update
MCQ — jeden poprawny	4 opcje a/b/c/d, jedna właściwa	Pełny punkt lub zero	Automatyczny (q=5 lub q=0)
MCQ — multi-correct	4 opcje, wiele poprawnych (multi_correct=True)	Pełny za komplet, częściowy za podzbiór	Automatyczny
OPEN — otwarte	Wpisanie odpowiedzi tekstem + wzorzec	Samoocena ☐ /△/ ☐	Na podstawie samooceny
CODE — egzaminator	Fragment kodu Python, analiza zachowania	MCQ lub samoocena	Automatyczny lub samoocena

Tryby quizu

Tryb	Opis	Wpływ na SRS	Specyfika
Ręczny (manual)	Pełna kontrola filtrów	Tak	Wybór lekcji, modułów, trudności, ilości
SRS / Powtórka (srs)	Algorytm SM-2 decyduje	Tak	Tylko pytania z next_review_date ≤ dziś
Egzaminacyjny (exam)	Symulacja egzaminu końcowego	Tak	100 pytań, opcjonalny timer, raport z prognozą
Flashcards	Szybka powtórka	Tak	Pytanie → odkrycie → samoocena ☐ / ☐
Błędy (mistakes)	Powtórka słabych pytań	Tak	Pytania z weak_questions (☐ +△)

Funkcja "Pomiń — wróć później"

Dostępna we wszystkich trybach poza egzaminacyjnym. Kliknięcie przycisku "Pomiń" przenosi pytanie na koniec kolejki sesji bez wpływu na SRS. Maksymalnie 3 pominięcia tego samego pytania w jednej sesji — przy czwartym jest wymagana odpowiedź.

10. Dashboard i analityka

Elementy dashboardu głównego

Powitanie z datą — Spersonalizowane powitanie z aktualną datą

Pasek do egzaminu — "X dni do egzaminu [data]" — widoczny gdy ustawiona data egzaminu

4 karty metryk — Streak · Pytania SRS dziś · Opanowane · Skuteczność 7 dni

Kolejka SRS — Lista najbliższych pytań do powtórki z lekcją i trudnością

Mapa opanowania — Siatka lekcji z kolorami: zielony/żółty/czerwony/szary

Rekomendacje — Max 3 rekomendacje: co powtórzyć i dlaczego

Cel egzaminacyjny — Pasek postępu + pytania dziennie + odznaki

Wykres aktywności — Słupki dla ostatnich 7 dni nauki

Mapa opanowania — progi kolorów

Kolor	Próg accuracy	Znaczenie
Zielony	$\geq 80\%$	Lekcja opanowana
Żółty	50–79%	Średni poziom opanowania
Czerwony	$< 50\%$	Wymaga intensywnej pracy
Szary	0%	Brak aktywności dla tej lekcji

System rekomendacji

Algorytm rekomendacji analizuje trzy sygnały i łączy je w priorytetową listę maksymalnie 3 sugestii dziennie:

- Pilność SRS — pytania z przeterminowaną `next_review_date`
- Słabe wyniki — lekcje z accuracy poniżej 60% w ostatnich 10 sesjach
- Nieaktywność — lekcje nierobione przez ponad 7 dni

Analityka szczegółowa (/progress)

- Krzywa nauki — wykres liniowy accuracy w czasie (7/30/90 dni)
- Statystyki per lekcja — tabela z % poprawnych, liczba prób, średni czas
- Statystyki per trudność — rozkład wyników easy/medium/hard/expert
- Statystyki czasowe — średni czas odpowiedzi per lekcja i temat
- Historia sesji — pełna lista quizów z wynikami, datami i trybem

Raport po egzaminie próbnym

- Wynik ogólny procentowy

- Wynik per moduł (breakdown)
- Top 5 najczęstszych błędów
- Porównanie z poprzednimi próbami (wykres trendów)
- Prognoza zdania: obliczana jako $\text{weighted_accuracy} \times \text{confidence_factor}$
- Pełna lista pytań z oznaczeniem błędnych

11. Mapa ekranów

/ (root) → redirect do /dashboard

/dashboard	← Strona główna aplikacji
/lessons	← Lista modułów i lekcji
/lessons/[id]	← Szczegóły lekcji: PDF viewer + pytania
/quiz/configure	← Konfiguracja quizu (filtry, tryb, ilość)
/quiz/[sessionId]	← Aktywny quiz
/quiz/[sessionId]/summary	← Podsumowanie i wyniki sesji
/srs	← Dzienna powtórka SRS
/exam	← Tryb egzaminacyjny (start + config)
/flashcards	← Tryb flashcard
/review	← Panel zatwierdzania importowanych pytań
/mistakes	← Pytania do poprawy (□ + ▲□)
/bookmarks	← Zakładki
/notes	← Notatki
/progress	← Analityka, krzywa nauki, raporty
/settings	← Data egzaminu, cel dzienny, profil

12. Docker i środowisko lokalne

docker-compose.yml

```
version: '3.9'

services:
  db:
    image: postgres:16-alpine
    volumes:
      - postgres_data:/var/lib/postgresql/data/
    environment:
      POSTGRES_DB: pyquiz
      POSTGRES_USER: pyquiz
      POSTGRES_PASSWORD: pyquiz_secret
    ports: ["5432:5432"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U pyquiz"]
      interval: 5s
      retries: 5

  backend:
    build: ./backend
    command: python manage.py runserver 0.0.0.0:8000
    volumes:
      - ./backend:/app
      - pdf_files:/app/media/pdfs
    ports: ["8000:8000"]
    environment:
      DATABASE_URL: postgres://pyquiz:pyquiz_secret@db:5432/pyquiz
      DEBUG: "True"
    depends_on:
      db:
        condition: service_healthy

  frontend:
    build: ./frontend
    command: npm run dev
    ports: ["3000:3000"]
    environment:
      NEXT_PUBLIC_API_URL: http://localhost:8000/api
    depends_on: [backend]
```

```
volumes:  
  postgres_data:  
  pdf_files:
```

Uruchomienie projektu

```
# 1. Klonowanie repozytorium  
git clone https://github.com/username/pyquiz.git && cd pyquiz  
  
# 2. Uruchomienie wszystkich serwisów  
docker-compose up --build  
  
# 3. Migracje bazy danych  
docker-compose exec backend python manage.py migrate  
  
# 4. Tworzenie użytkownika  
docker-compose exec backend python manage.py createsuperuser  
  
# 5. Seed danych startowych (odznaki, domyślne ustawienia)  
docker-compose exec backend python manage.py seed_data  
  
# Dostępne adresy:  
#   Frontend:      http://localhost:3000  
#   Backend API:   http://localhost:8000/api  
#   Django Admin: http://localhost:8000/admin
```


13. Struktura projektu

```
pyquiz/
├─ backend/
│   ├─ apps/
│   │   ├─ analytics/      ← Dashboard, statystyki, raporty
│   │   ├─ lessons/       ← Moduły, lekcje, PDF
│   │   ├─ questions/     ← Bank pytań, import, review
│   │   ├─ quiz/          ← Sesje quizów, odpowiedzi
│   │   ├─ srs/           ← Algorytm SM-2
│   │   └─ users/         ← Użytkownicy, autoryzacja
│   ├─ config/
│   │   ├─ settings/
│   │   │   └─ base.py
│   │   │   └─ local.py
│   │   └─ urls.py
│   ├─ scripts/
│   │   └─ import_questions.py
│   ├─ media/pdfs/        ← Pliki PDF lekcji
│   ├─ Dockerfile
│   ├─ manage.py
│   └─ requirements.txt
├─ frontend/
│   ├─ app/               ← Next.js App Router
│   ├─ components/       ← Komponenty React
│   ├─ hooks/            ← Custom hooks
│   ├─ lib/              ← API client, typy TypeScript
│   ├─ public/
│   ├─ Dockerfile
│   └─ package.json
├─ imports/              ← Katalog na JSON z pytaniami
├─ docker-compose.yml
├─ .env.example
├─ .gitignore
└─ README.md
```

14. Plan implementacji

Etap 1

Fundament

Tydzień 1-2

- ☐ Setup repozytorium Git + Docker + docker-compose
- ☐ Projekt Django z konfiguracją settings (base/local)
- ☐ Wszystkie modele Django + migracje bazy danych
- ☐ Django Admin dla zarządzania danymi
- ☐ Skrypt import_questions.py (CLI)
- ☐ Podstawowe API endpoints (lessons, questions, modules)
- ☐ Setup projektu Next.js + Tailwind CSS + shadcn/ui
- ☐ Sidebar + podstawowy routing Next.js

Etap 2

Quiz Core

Tydzień 3-4

- ☐ API: tworzenie sesji quizu z filtrami
- ☐ API: pobieranie pytań, zapisywanie odpowiedzi
- ☐ Frontend: ekran konfiguracji quizu
- ☐ Frontend: komponent MCQ (jedno i wielokrotny wybór)
- ☐ Frontend: komponent pytania otwartego z samooceną
- ☐ Frontend: wyjaśnienia po błędnej odpowiedzi
- ☐ Frontend: podsumowanie sesji z wynikami
- ☐ Panel review pytań (zatwierdzanie importów)

Etap 3

SRS i tryby zaawansowane

Tydzień 5

- ☐ Implementacja algorytmu SM-2 (calculate_sm2)
- ☐ Automatyczne tworzenie SRSCard po imporcie pytań
- ☐ API: dzienna kolejka SRS, aktualizacja kart po odpowiedzi
- ☐ Frontend: tryb dzienna powtórka (SRS)
- ☐ Frontend: tryb egzaminacyjny z timerem
- ☐ Frontend: flashcards
- ☐ Frontend: tryb "powtórka błędów"
- ☐ Funkcja "Pomiń — wróć później" z powrotem na koniec kolejki

Etap 4

Dashboard i analityka

Tydzień 6-7

- ☐ API: wszystkie metryki dashboardu
- ☐ API: mapa opanowania (accuracy per lekcja)
- ☐ API: algorytm rekomendacji
- ☐ Frontend: dashboard główny z kartami metryk
- ☐ Frontend: mapa opanowania (Knowledge Map)
- ☐ Frontend: wykresy postępów Recharts

- ☐ Frontend: strona analityki szczegółowej /progress
- ☐ Streak + system odznak (badges)
- ☐ Raport tygodniowy
- ☐ Raport po egzaminie próbnym z prognozą zdania

**Etap
5****Funkcje dodatkowe i dopracowanie**

Tydzień 8

- ☐ PDF viewer w aplikacji (react-pdf)
- ☐ Kontekst pytania — "Pokaż fragment PDF źródłowy"
- ☐ Zakładki (bookmarks) i notatki podczas quizu
- ☐ Statystyki czasowe per temat
- ☐ Panel importu JSON w UI (alternatywa dla CLI)
- ☐ Tryb "Tylko teoria" — Recap (streszczenia lekcji)
- ☐ Cel egzaminacyjny z licznikiem dni i obliczaniem tempa
- ☐ Ustawienia użytkownika (data egzaminu, cel dzienny)

15. Format JSON pytań

Poniżej pełna specyfikacja formatu JSON generowanego przez Claude AI dla każdej lekcji. Plik należy zapisać z nazwą opisującą lekcję, np. `lekcja_14_dekoratory.json`.

```
{
  "module": {
    "number": 3,
    "title": "Zaawansowany Python"
  },
  "lesson": {
    "number": 14,
    "title": "Dekoratory w Pythonie",
    "summary": "Streszczenie lekcji generowane przez Claude..."
  },
  "questions": [
    {
      "type": "MCQ",
      "difficulty": "medium",
      "multi_correct": false,
      "topic_tags": ["dekoratory", "functools"],
      "question_text": "Co jest wymagane aby dekorator nie ukrywał metadanych?",
      "choices": [
        {"label": "a", "text": "Użycie @staticmethod", "is_correct": false},
        {"label": "b", "text": "Użycie functools.wraps", "is_correct": true},
        {"label": "c", "text": "Zwrócenie oryginalnej funkcji", "is_correct": false},
        {"label": "d", "text": "Deklaracja jako metoda klasy", "is_correct": false}
      ],
      "correct_answer": "b",
      "explanation": "functools.wraps(func) kopiuje metadane...",
      "source_context": "Fragment PDF z którego pochodzi pytanie..."
    },
    {
      "type": "MCQ",
      "difficulty": "hard",
      "multi_correct": true,
      "topic_tags": ["dekoratory", "closures"],
      "question_text": "Które stwierdzenia o dekoratorach z argumentami są prawdziwe?",
      "choices": [
        {"label": "a", "text": "Wymagają dodatkowej warstwy zagnieżdżenia", "is_correct": true},
        {"label": "b", "text": "Mogą być wywoływane bez nawiasów", "is_correct": false},

```

```
{
  "label": "c", "text": "Zwracają dekorator, który zwraca wrapper", "is_correct": true},
  {"label": "d", "text": "Nie mogą używać functools.wraps", "is_correct": false}
],
"correct_answer": "a,c",
"explanation": "Dekorator z argumentami to fabryka dekoratorów...",
"source_context": ""
},
{
  "type": "OPEN",
  "difficulty": "hard",
  "multi_correct": false,
  "topic_tags": ["dekoratory", "praktyczne"],
  "question_text": "Napisz dekorator @retry(times=3) który ponawia wywołanie przy wyjątku.",
  "choices": [],
  "correct_answer": "def retry(times=3):\n    def decorator(func):\n        ...",
  "explanation": "Kluczowe elementy: zewnętrzna funkcja retry, pętla prób...",
  "source_context": ""
},
{
  "type": "CODE",
  "difficulty": "expert",
  "multi_correct": false,
  "topic_tags": ["dekoratory", "debugowanie"],
  "question_text": "Co wypisze poniższy kod?\n\n@my_decorator\ndef greet(): ...\nprint(greet.__name__)",
  "choices": [],
  "correct_answer": "wrapper",
  "explanation": "Bez functools.wraps metadane są nadpisane przez wrapper...",
  "source_context": ""
}
]
```

16. API Endpoints

Moduły i lekcje

Metoda	Endpoint	Opis
GET	/api/modules/	Lista wszystkich modułów
GET	/api/modules/{id}/	Szczegóły modułu
GET	/api/lessons/	Lista wszystkich lekcji
GET	/api/lessons/{id}/	Szczegóły lekcji
GET	/api/lessons/{id}/pdf/	Serwowanie pliku PDF

Pytania

Metoda	Endpoint	Opis
GET	/api/questions/	Lista pytań (filtry: lesson_id, module_id, difficulty, type, tags)
GET	/api/questions/{id}/	Szczegóły pytania
PATCH	/api/questions/{id}/	Edycja pytania
DELETE	/api/questions/{id}/	Usunięcie pytania
POST	/api/questions/{id}/approve/	Zatwierdzenie pytania (review)
GET	/api/questions/pending/	Pytania oczekujące na review

Quizy

Metoda	Endpoint	Opis
POST	/api/quiz/sessions/	Tworzenie nowej sesji quizu
GET	/api/quiz/sessions/{id}/	Pobieranie sesji z pytaniami
POST	/api/quiz/sessions/{id}/answer/	Zapisanie odpowiedzi
POST	/api/quiz/sessions/{id}/finish/	Zakończenie sesji
GET	/api/quiz/sessions/{id}/summary/	Podsumowanie i wyniki
GET	/api/quiz/history/	Historia wszystkich sesji

SRS i analityka

Metoda	Endpoint	Opis
GET	/api/srs/due/	Pytania do powtórki dziś
GET	/api/srs/stats/	Statystyki SRS
GET	/api/srs/forecast/	Prognoza ilości powtórek (14 dni)
GET	/api/dashboard/	Wszystkie dane dashboardu
GET	/api/analytics/progress/	Krzywa nauki (param: days=7/30/90)
GET	/api/analytics/lessons/	Statystyki accuracy per lekcja
GET	/api/analytics/time/	Statystyki czasowe per temat
GET	/api/analytics/weak/	Lista pytań do poprawy
GET	/api/analytics/weekly-report/	Raport tygodniowy

Zakładki, notatki i import

Metoda	Endpoint	Opis
GET	/api/bookmarks/	Lista zakładek
POST	/api/bookmarks/	Dodanie zakładki
DELETE	/api/bookmarks/{question_id}/	Usunięcie zakładki
GET	/api/notes/	Lista notatek
POST	/api/notes/	Dodanie/aktualizacja notatki
DELETE	/api/notes/{question_id}/	Usunięcie notatki
POST	/api/import/json/	Import pytań z pliku JSON
GET	/api/import/history/	Historia importów

17. Słownik pojęć

Pojęcie	Definicja
SRS	Spaced Repetition System — system powtórek rozłożonych w czasie, pokazuje trudniejszy materiał częściej
SM-2	SuperMemo 2 — algorytm SRS opracowany przez Piotra Woźniaka, używany m.in. w Anki
SRSCard	Rekord w bazie przechowujący stan algorytmu SM-2 dla pary (użytkownik, pytanie)
Interval	Liczba dni do następnej zaplanowanej powtórki danego pytania
Ease Factor (EF)	Współczynnik łatwości pytania — im wyższy, tym rzadziej pytanie będzie pokazywane
MCQ	Multiple Choice Question — pytanie z wyborem jednej lub wielu odpowiedzi z listy
Panel Review	Ekran do zatwierdzania importowanych pytań przed dopuszczeniem do quizów
Streak	Liczba kolejnych dni aktywnej nauki (jak w Duolingo)
Knowledge Map	Wizualna siatka lekcji z kolorowym oznaczeniem poziomu opanowania materiału
Weak Questions	Pytania na których użytkownik popełnił błędy (□ lub △), trafiają do sekcji "Do poprawy"
Flashcard	Tryb szybkiej powtórki: pytanie → kliknięcie odkrywa odpowiedź → samoocena □/□
Recap	Streszczenie lekcji generowane przez Claude razem z pytaniami, dostępne przed quizem
Egzaminator	Specjalny typ pytania CODE — analiza fragmentu kodu Python (co robi / jaki błąd zwróci)
Prognoza zdania	Szacunkowy procent szans na zdanie egzaminu, obliczany z wyników egzaminów próbnych
multi_correct	Flaga na pytaniu MCQ oznaczająca, że może być więcej niż jedna poprawna odpowiedź
is_approved	Flaga na pytaniu oznaczająca, że przeszło panel review i może być użyte w quizach

PyQuiz v1.0 · Dokumentacja Projektu · Kwiecień 2026

Aplikacja do nauki Python + Django z algorytmem Spaced Repetition (SM-2).
Zbudowana w Django REST Framework + Next.js 14 + PostgreSQL + Docker.

Autor: Krystian